

JavaScript Funktionen

GRG, HOA, WZR

Web- and Mobile Computing

12. März 2026

Funktion

Funktionen...

- dienen der Mehrfachverwendung und Gliederung von Code.
- können eine (fast) beliebige Anzahl von Parametern bekommen.
- können einen Rückgabewert haben (aber müssen nicht).
- Innerhalb der Funktion besteht Zugriff auf den globalen Scope (Variablen, welche mit 'const' oder 'let' deklariert wurden)
- Eine Funktion kann selber Parameter für eine andere Funktion sein. (callbacks, wie z.B. 'setTimeout(function, milliseconds)')
- Funktionen können einen Namen haben, aber auch *anonym* sein (Lambda-Functions, bzw. Arrow-Functions)

Funktion

```
1  function begruessung(name) {  
2      return "Hallo_" + name;  
3  }  
4  
5  let text = begruessung("Anna");  
6  console.log(text);
```

Erklärung:

- function startet die Funktionsdefinition
- name ist ein Parameter
- return gibt das Ergebnis zurück

Funktion

Funktionen können auch in Variablen gespeichert werden:

```
1      const addiere = function(a, b) {  
2          return a + b;  
3      };  
4  
5      console.log(addiere(3, 4));
```

Wichtig:

- Funktionen sind in JavaScript **Werte**
- Man kann sie wie Zahlen oder Strings speichern

Methoden

Eine Methode ist eine Funktion, die zu einem Objekt gehört.

```
1  const student = {  
2      name: "Tom",  
3      sageHallo: function() {  
4          return "Hallo, ich bin " + this.name;  
5      }  
6  };  
7  
8  console.log(student.sageHallo());
```

- Eine Methode ist eine Funktion in einem Objekt
- `this` bezieht sich auf das Objekt

Arrow-Functions / Lambdas

Seit ES6 gibt es eine kürzere Schreibweise:

- Weniger Schreibarbeit
- Übersichtlicher Code
- Besonders praktisch bei kurzen Funktionen

```
1   const addiere = (a, b) => {  
2       return a + b;  
3   };
```

Noch kürzer (wenn nur eine Zeile):

```
const addiere = (a, b) => a + b;
```

Arrow-Functions / Lambdas

- Links stehen die Parameter
- Rechts steht die Berechnung
- Das Wort `function` entfällt
- Die Funktion hat keinen Namen

Vergleich:

- Normal: `function(a, b) { return a + b; }`
- Arrow: `(a, b) => a + b`

Beispiel: Array sortieren mit Arrow Function

```
1  const zahlen = [5, 2, 9, 10, 7];  
2  
3  zahlen.sort((a, b) => a - b);  
4  
5  console.log(zahlen);
```

Was passiert hier?

- `sort()` sortiert ein Array
- Die Funktion `(a, b) => a - b` sagt, **wie** sortiert werden soll
- Wenn das Ergebnis negativ ist $\rightarrow$ a kommt vor b
- Wenn das Ergebnis positiv ist $\rightarrow$ b kommt vor a
- Wenn das Ergebnis 0 ist $\rightarrow$ Reihenfolge bleibt gleich

Warum brauchen wir das?

Ohne diese Funktion würde JavaScript Zahlen wie Texte sortieren:
[10, 2, 5, 7, 9]

Beispiel: Alert

Variante 1 (falsch):

```
setTimeout(alert("Hello_World"), 3000);
```

- `alert("Hello World")` wird **sofort** ausgeführt
- Das Ergebnis (`undefined`) wird an `setTimeout` übergeben
- Es gibt **keine Verzögerung**

Variante 2 (richtig):

```
setTimeout(() => alert("Hello_World"), 3000);
```

- Eine Funktion wird übergeben
- Diese Funktion wird erst nach 3000 ms ausgeführt
- Der Alert erscheint also nach 3 Sekunden

Mit `()` wird eine Funktion **aufgerufen**.

Ohne `()` wird eine Funktion **übergeben**.

Parameter/Argumente

Für die Sprache selbst ist es egal, ob der Aufrufer die “richtige” Anzahl an Parametern mitgibt. Beispiel ohne Parameter, aber dennoch werden welche übergeben:

```
function f() {  
    for (let i = 0; i < arguments.length; i++) {  
        console.log(i, arguments[i]);  
    }  
}  
f(23, "georg", new Date());
```

Output:

```
0 23  
1 georg  
2 2023-10-18T20:28:55.244Z
```

Parameter/Argumente

Hier werden “zu wenige” Parameter übergeben, siehe ‘undefined’ im Output:

```
1      function f(a, b, c) {  
2          console.log(a);  
3          console.log(b);  
4          console.log(c);  
5          for (let i = 0; i < arguments.length; i++) {  
6              console.log(i, arguments[i]);  
7          }  
8      }  
9      f(23, 3.14159);
```

Ausgabe:

```
1      23  
2      3.14159  
3      undefined  
4      0 23  
5      1 3.14159
```

Kein Rückgabewert

Gleiches gilt für den Rückgabewert einer Funktion:

```
1      function f() {}  
2      y = f();  
3      console.log(y);
```

Ausgabe:

```
undefined
```

Eine Funktion als Argument

```
1      function f(funcPar, actualPar) {  
2          funcPar("The answer:", actualPar);  
3      }  
4      f(console.log, 42);
```

Der Parameter ist hier die 'console.log'.

Ausgabe:

```
The answer: 42
```

Closures (Grundidee)

- Eine **Closure** ist eine Funktion + ihr umgebender Scope
- Die innere Funktion merkt sich Variablen der äußeren Funktion
- Das funktioniert auch dann, wenn die äußere Funktion schon beendet ist

```
1  function makeGreeter(name) {  
2      return function() {  
3          return "Hallo_" + name;  
4      };  
5  }  
6  
7  const greetAnna = makeGreeter("Anna");  
8  console.log(greetAnna()); // Hallo Anna
```

Closures (Beispiel: Counter)

```
1      function makeCounter() {  
2          let count = 0;  
3  
4          return () => {  
5              count++;  
6              return count;  
7          };  
8      }  
9  
10     const counter1 = makeCounter();  
11     const counter2 = makeCounter();  
12  
13     console.log(counter1()); // 1  
14     console.log(counter1()); // 2  
15     console.log(counter2()); // 1
```

- Jeder Aufruf von `makeCounter()` erzeugt eine eigene Closure
- `count` ist von außen nicht direkt zugreifbar